RES POLYTECHNIC, BANGALORE-91

CIE Practice Test IV - Model Answers

Programming Concepts in Mechanical Engineering | Course Code 25ME34I | Marks 50 | Semester III

Answer any one full question (Q1 or Q2). Both are answered below. Where data is not given in the question, sample data is assumed.

Q1

a) Mean, variance and standard deviation of shaft diameters (5 marks)

```
import numpy as np

d = [24.9, 25.1, 25.0, 25.2, 24.8, 25.0, 25.1, 24.9, 25.0, 25.0]

mean = np.mean(d)
var = np.var(d, ddof=1)       # sample variance
sd = np.std(d, ddof=1)        # sample standard deviation

print("Mean =", mean)
print("Variance =", round(var, 4))
print("Std Dev =", round(sd, 4))
```

Output: Mean = 25.0 mm, Variance = 0.0133 mm², Standard deviation = 0.1155 mm

(Check: $\Sigma(x - \mu)^2 = 0.12$; variance = 0.12/9 = 0.0133.)

b) In-control and out-of-control samples (5 marks)

```
avg = [10.02, 10.05, 9.98, 10.10, 10.01, 9.90, 10.03]
UCL, LCL = 10.08, 9.92

for i, x in enumerate(avg, start=1):
    if LCL <= x <= UCL:
        print("Sample", i, x, "-> In control")
    else:
        print("Sample", i, x, "-> Out of control")
```

Result: Sample 4 (10.10 > UCL) and Sample 6 (9.90 < LCL) are out of control; samples 1, 2, 3, 5 and 7 are in control.

c) X-bar and R charts using Matplotlib (10 marks)

```
import numpy as np
import matplotlib.pyplot as plt

data = np.array([[25.0, 25.1, 24.9, 25.0],
                 [25.2, 25.0, 25.1, 24.9],
                 [24.8, 25.0, 25.1, 25.0],
                 [25.1, 25.3, 25.0, 25.2],
                 [24.9, 25.0, 24.8, 25.1]])

xbar = data.mean(axis=1)                    # sample average
R = data.max(axis=1) - data.min(axis=1)     # sample range
xbb, Rb = xbar.mean(), R.mean()
A2, D3, D4 = 0.729, 0, 2.282                # constants for n = 4

print("X-bar =", xbar)
print("R     =", R)

fig, ax = plt.subplots(2, 1, figsize=(8, 7))
ax[0].plot(xbar, marker='o')
ax[0].axhline(xbb, color='g', label='CL')
ax[0].axhline(xbb + A2*Rb, color='r', linestyle='--', label='UCL')
ax[0].axhline(xbb - A2*Rb, color='r', linestyle='--', label='LCL')
ax[0].set_title("X-bar chart"); ax[0].legend()

ax[1].plot(R, marker='o')
ax[1].axhline(Rb, color='g', label='CL')
ax[1].axhline(D4*Rb, color='r', linestyle='--', label='UCL')
ax[1].axhline(D3*Rb, color='r', linestyle='--', label='LCL')
ax[1].set_title("R chart"); ax[1].legend()
plt.tight_layout(); plt.show()
```

Limits: X-bar chart: UCL = $\bar{\bar{X}} + A2 \cdot \bar{R}$, LCL = $\bar{\bar{X}} - A2 \cdot \bar{R}$. R chart: UCL = $D4 \cdot \bar{R}$, LCL = $D3 \cdot \bar{R}$.

d) DC motor (forward, reverse, stop) with L298N and Raspberry Pi (15 marks)

Connections: GPIO 17 → IN1, GPIO 27 → IN2, GPIO 22 → ENA, Pi GND → L298N GND, motor → OUT1/OUT2, external 12 V supply → 12V terminal.

```
import RPi.GPIO as GPIO
import time

IN1, IN2, ENA = 17, 27, 22
GPIO.setmode(GPIO.BCM)
GPIO.setup([IN1, IN2, ENA], GPIO.OUT)
GPIO.output(ENA, GPIO.HIGH)          # enable motor

def forward():
    GPIO.output(IN1, GPIO.HIGH)
    GPIO.output(IN2, GPIO.LOW)

def reverse():
    GPIO.output(IN1, GPIO.LOW)
    GPIO.output(IN2, GPIO.HIGH)

def stop():
    GPIO.output(IN1, GPIO.LOW)
    GPIO.output(IN2, GPIO.LOW)

try:
    forward(); print("Forward"); time.sleep(3)
    stop();    print("Stop");    time.sleep(1)
    reverse(); print("Reverse"); time.sleep(3)
    stop();    print("Stop")
finally:
    GPIO.cleanup()
```

IN1	IN2	Motor
HIGH	LOW	Forward
LOW	HIGH	Reverse
LOW	LOW	Stop

e) Stress distribution in a plate/bar along its length (15 marks)

Axial stress at any section: $\sigma = P / A(x) = P / (w(x) \cdot t)$. A stepped plate is taken: P = 10 kN, thickness t = 5 mm, width 60 mm for the first half and 40 mm for the second half (length 200 mm).

```
import numpy as np
import matplotlib.pyplot as plt

P = 10000            # N
t = 5                # mm
L = 200              # mm

x = np.linspace(0, L, 400)
w = np.where(x <= 100, 60, 40)     # width (mm) at each x
sigma = P / (w * t)                # stress (MPa)

print("Stress in 60 mm section =", P/(60*t), "MPa")   # 33.33
print("Stress in 40 mm section =", P/(40*t), "MPa")   # 50.0

plt.plot(x, sigma)
plt.xlabel("Length along bar x (mm)")
plt.ylabel("Stress (MPa)")
plt.title("Stress distribution along the plate")
plt.grid(True)
plt.show()
```

Stress is lower where the plate is wider (33.3 MPa) and higher where it is narrower (50 MPa).

OR

Q2

a) Process capability Cp and Cpk (5 marks)

```
mean, sd = 50.02, 0.15
USL, LSL = 50.5, 49.5

Cp = (USL - LSL) / (6 * sd)
Cpk = min((USL - mean) / (3 * sd), (mean - LSL) / (3 * sd))

print("Cp  =", round(Cp, 3))
```

```
print("Cpk =", round(Cpk, 3))
```

Calculation: Cp = 1.0 / (6 × 0.15) = **1.11**; Cpk = min(0.48/0.45, 0.52/0.45) = min(1.07, 1.16) = **1.07**. Since Cpk < 1.33, the process needs improvement (it is slightly off-centre).

b) Sample average and range of five samples of size four (5 marks)

```
import numpy as np

samples = np.array([[10.1, 10.0, 9.9, 10.0],
                    [10.2, 10.1, 10.0, 10.1],
                    [9.9, 10.0, 10.1, 10.0],
                    [10.0, 10.2, 10.1, 9.9],
                    [10.1, 10.0, 10.0, 10.1]])

for i, s in enumerate(samples, start=1):
    print("Sample", i, "Average =", round(s.mean(), 3),
          "Range =", round(s.max() - s.min(), 3))
```

Average = sum of readings / 4; Range = largest – smallest reading.

c) Pareto chart and histogram (10 marks)

```
import numpy as np
import matplotlib.pyplot as plt

# Pareto chart
labels = ["Blowholes", "Cracks", "Misruns", "Shrinkage", "Others"]
count = np.array([40, 25, 15, 10, 10])
cum = np.cumsum(count) / count.sum() * 100

fig, ax = plt.subplots()
ax.bar(labels, count)
ax.set_ylabel("Defect count")
ax2 = ax.twinx()
ax2.plot(labels, cum, color='r', marker='o')
ax2.set_ylabel("Cumulative %")
plt.title("Pareto chart")
plt.show()

# Histogram
dia = [24.9, 25.1, 25.0, 25.2, 24.8, 25.0, 25.1, 24.9, 25.0, 25.0,
       25.1, 24.9, 25.0, 25.0, 25.2]
plt.hist(dia, bins=5, edgecolor='black')
plt.xlabel("Shaft diameter (mm)")
plt.ylabel("Frequency")
plt.title("Histogram")
plt.show()
```

Pareto chart: bars in descending order plus a cumulative-% line; it identifies the "vital few" defects.
Histogram: shows the frequency distribution and spread of the data.

d) Automated water pump controller (15 marks)

Connections: Level sensor OUT → GPIO 17 (input; LOW = 0 when tank is low, FULL = 1 when tank is full). Pump connects through a relay module; relay IN → GPIO 27. Relay VCC → 5 V, GND → GND.

```
import RPi.GPIO as GPIO
import time

SENSOR = 17
RELAY = 27

GPIO.setmode(GPIO.BCM)
GPIO.setup(SENSOR, GPIO.IN)
GPIO.setup(RELAY, GPIO.OUT)

try:
    while True:
        level = GPIO.input(SENSOR)
        if level == 0:                          # LOW level
            GPIO.output(RELAY, GPIO.HIGH)       # pump ON
            print("Tank LOW  -> Pump ON")
        else:                                   # FULL
            GPIO.output(RELAY, GPIO.LOW)        # pump OFF
            print("Tank FULL -> Pump OFF")
        time.sleep(1)
except KeyboardInterrupt:
    pass
finally:
    GPIO.cleanup()
```

e) Stress distribution in a plate with and without a hole (15 marks)

Data: P = 10 kN, plate width w = 50 mm, thickness t = 5 mm, hole diameter d = 10 mm at the centre (x = 100 mm), length 200 mm.

- **Without hole:** σ = P / (w·t) = 10000 / 250 = **40 MPa** (uniform).
- **With hole:** net-section stress σn = P / ((w – d)·t) = 10000 / 200 = **50 MPa**; maximum stress at the hole edge = Kt × σn, where Kt = 3 – 3.14(d/w) + 3.667(d/w)² – 1.527(d/w)³ ≈ 2.51 → σmax ≈ **125 MPa**.

```
import numpy as np
import matplotlib.pyplot as plt

P, w, t, d, L = 10000, 50, 5, 10, 200

x = np.linspace(0, L, 400)

# Without hole
sigma_plain = np.full_like(x, P / (w * t))

# With hole (hole region 95 to 105 mm)
sigma_hole = np.full_like(x, P / (w * t))
hole = (x >= 95) & (x <= 105)
sigma_hole[hole] = P / ((w - d) * t)          # net-section stress

r = d / w
Kt = 3 - 3.14*r + 3.667*r**2 - 1.527*r**3
print("Nominal stress   =", P/(w*t), "MPa")
print("Net stress       =", P/((w-d)*t), "MPa")
print("Kt =", round(Kt, 2), " Max stress =", round(Kt*P/((w-d)*t), 1), "MPa")

plt.plot(x, sigma_plain, label="Without hole")
plt.plot(x, sigma_hole, label="With hole (net section)")
plt.xlabel("Length x (mm)")
plt.ylabel("Stress (MPa)")
plt.title("Stress distribution: with and without hole")
plt.legend(); plt.grid(True); plt.show()
```

Conclusion: a hole reduces the area, raising stress at that section (40 → 50 MPa net) and causes stress concentration (peak ≈ 125 MPa) at the hole edge.

Note: Data/values marked "assumed" are sample values; in the exam, use the data given in the question and write the final answer with units.